Decision Trees

Classification • Entropy & Gini • Information Gain • MATLAB + Python (Tutorial)

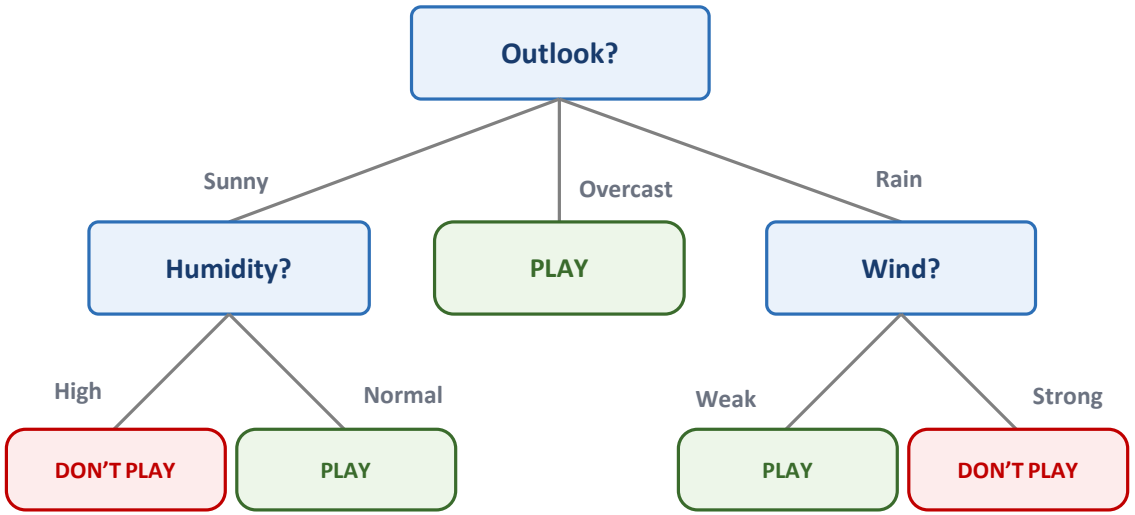
Presented by

Dibakar Roy

Supervisor: Dr. Avishek Adhikary

Department of Electrical Engineering, IIT Bhilai

Decision Tree anatomy



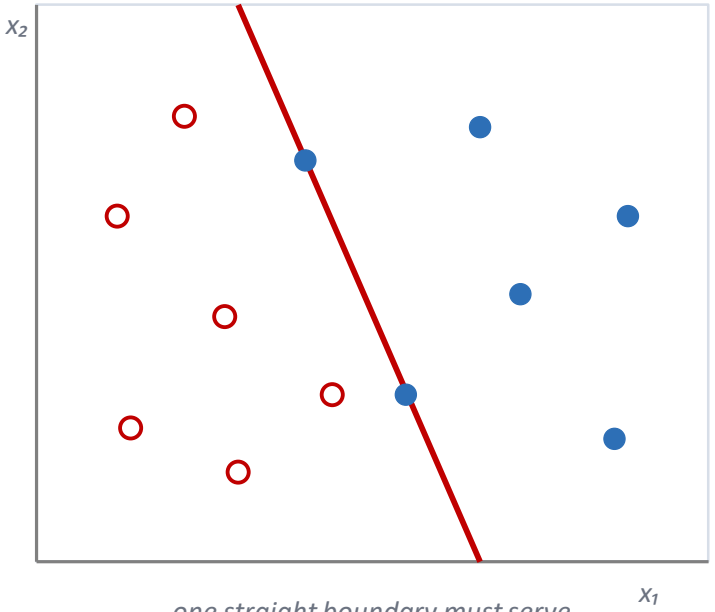
Tree Based on Play Tennis Dataset

Root node	the very first question — Outlook?
Internal node	any later question — Humidity?, Wind?
Branch	one outcome of a question — Sunny
Leaf node	a final answer — PLAY / DON'T PLAY
Split	dividing the examples into groups

Five words: root, internal node, branch, leaf, split. Everything else today is about choosing which question goes at each node.

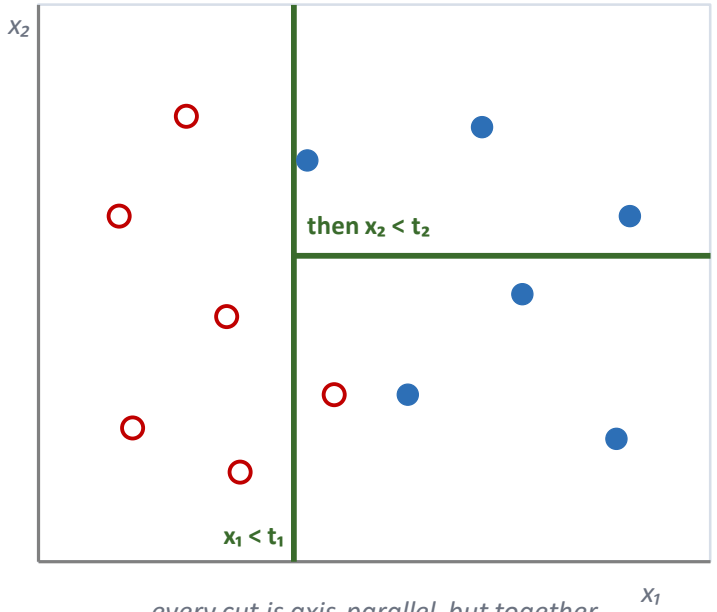
Why use a tree? Piecewise, non-linear decision regions

Linear model — one global rule



one straight boundary must serve the whole feature space

Decision tree — local rules



every cut is axis-parallel, but together they carve out non-linear regions

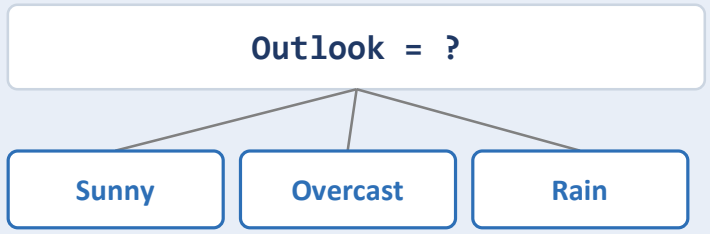
Observations

- A linear model commits to one global equation for the entire input space.
- A tree says: first check x_1, and only then decide what to check next.
- Different regions of the space can follow completely different rules.
- Every rule is a readable if-then sentence, so the model explains itself.

The tree never draws a diagonal line. It gets non-linearity from many simple cuts, not from one complicated equation.

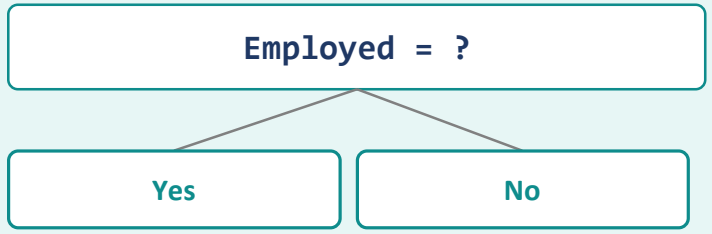
The questions a tree is allowed to ask

Categorical



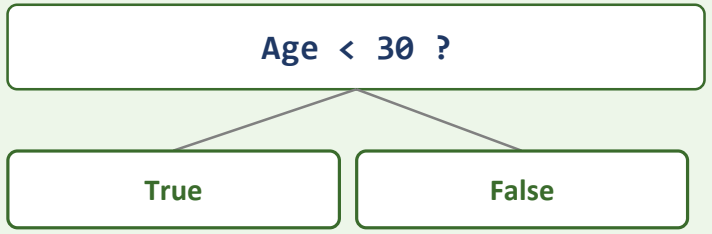
One branch per category. Three values → a three-way split.

Binary



The simplest case — exactly two branches, nothing to choose.

Continuous



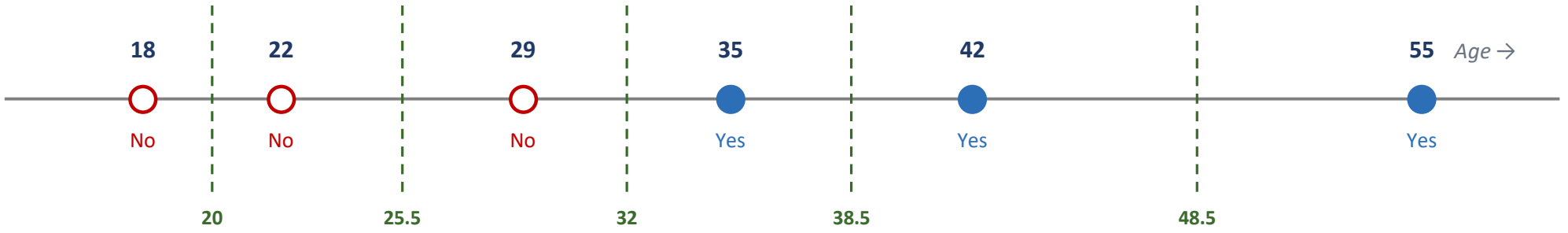
Always split into two. But which threshold? That is a real decision.

Where the real work is

A categorical feature offers one obvious split. A continuous feature offers a different split for every threshold you could pick — and the algorithm must try them.

The central learning problem: out of all the questions we could ask, which one gives the best split?

Continuous features: where does the threshold come from?



5 candidate thresholds, not infinitely many

The procedure

- Sort the distinct values of the feature.
- Take the midpoint between each neighbouring pair.
- For each midpoint, split the data in two and score the split.
- Keep the threshold with the best score; discard the rest.

Why midpoints, and why only these

Any threshold between 29 and 35 produces exactly the same two groups, so testing 30, 32 and 34 is wasted work.

n distinct values → at most n − 1 useful thresholds.

Here 32 separates the classes perfectly — but the tree only knows that after scoring it.

A good split makes the groups purer

Pure node



all five examples share one class
nothing left to ask

Mixed node



three of one class, two of the other
a question could still help

Maximally mixed



a 50/50 node — the worst case
a coin flip would do as well

What we need from a purity measure

Zero when pure

All examples one class → the measure must read 0.

Maximum when balanced

A 50/50 node is the most uncertain → highest value.

Symmetric

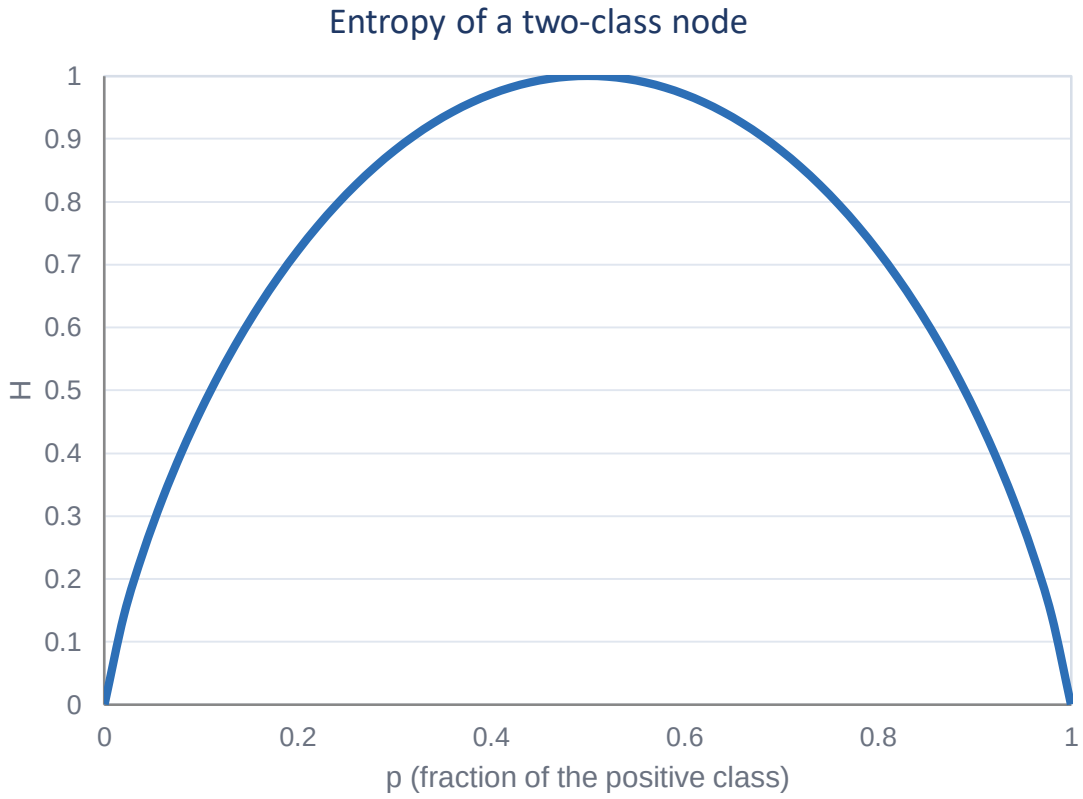
90% Yes and 90% No are equally pure — the labels are just names.

Entropy: putting ‘mixedness’ into one number

$$H(S) = -\sum p_i \log_2 (p_i)$$

p_i is the fraction of examples in class i at this node

100% PLAY	`H = 0.000`	*perfectly pure*
80% / 20%	`H = 0.722`	*mostly one class*
50% / 50%	`H = 1.000`	*maximally mixed*



Entropy = 0 means the node is finished. Entropy = 1 means the node tells you nothing. Every split we make is an attempt to move nodes towards 0.

Entropy by hand: one node, one calculation

The node

9 examples



6 PLAY **3 DON'T PLAY**

H is between 0 and 1 — but where?

Give students two minutes

Let them substitute the probabilities themselves. If the class is programming-focused, put $\log_2(2/3) = -0.585$ and $\log_2(1/3) = -1.585$ on the board so the arithmetic does not become the obstacle.

1 **Class proportions**

$p(\text{PLAY}) = 6/9 = 0.667$
$p(\text{DON'T}) = 3/9 = 0.333$

2 **Substitute into the formula**

$$H = -(0.667)\log_2(0.667) - (0.333)\log_2(0.333)$$

3 **Evaluate the logarithms**

$\log_2(0.667) = -0.585$
$\log_2(0.333) = -1.585$

4 **Combine**

$H = 0.667(0.585) + 0.333(1.585)$
$H = 0.390 + 0.528$

H(S)

0.918

quite mixed, but not 50/50

0.918 is high because the node is still badly mixed. The whole point of the next split is to drive this number down.

Gini impurity: what your code will actually use by default

$$\text{Gini}(S) = 1 - \sum p_i^2$$

the chance of mislabelling a random example if you guessed by the node's own class proportions

The same 6 / 3 node, in Gini

```
Gini = 1 - (6/9)² - (3/9)²
     = 1 - 0.444 - 0.111
     = 0.444
```

No logarithms — just squares. That is why it is the default: it is cheaper to compute, on every candidate split, at every node.

Entropy

`0 → 1`

$\log_2$ based • ID3, C4.5

Gini

`0 → 0.5`

squares only • CART, sklearn default

Watch the ceiling

A 50/50 node scores 1.0 in entropy but only 0.5 in Gini. The numbers are not comparable — only their ordering is.

sklearn's DecisionTreeClassifier uses Gini unless you ask for entropy

When you read gini = 0.0 in a plotted sklearn tree on Day 2, it means exactly what H = 0 meant here: a pure leaf.

Entropy vs Gini: does the choice change the tree?



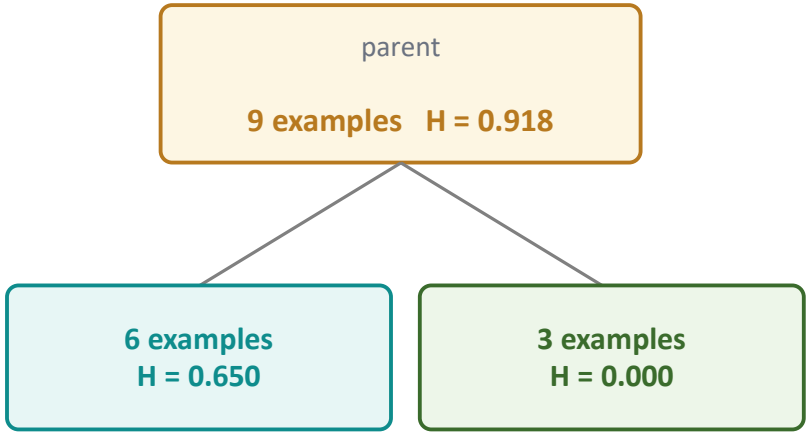
	Entropy	Gini
Pure node	0.0	0.0
50 / 50 node	1.0	0.5
Our 6 / 3 node	0.918	0.444
Cost to compute	logarithms	squares
Used by	ID3, C4.5	CART, sklearn
Set it with	`criterion='entropy'`	`criterion='gini'`

So which should you teach?

Both curves peak in the middle and vanish at the edges, so they rank candidate splits in nearly the same order. Compute entropy by hand because the $\log_2$ arithmetic makes the idea concrete — then let the library run Gini.

Information Gain: how much disorder did the split remove?

$$\text{Information Gain} = H(\text{parent}) - \sum \left(n_{\text{child}} / n_{\text{parent}} \right) \cdot H(\text{child})$$



Working it through

weighted child impurity

```
= (6/9)(0.650) + (3/9)(0.000)
= 0.433

IG = 0.918 − 0.433
   = 0.485
```

Why weighted?

A pure group of 3 helps less than a pure group of 300. Weighting by $n_{\text{child}} / n_{\text{parent}}$ stops the tree from chasing tiny, accidentally-pure branches.

Information Gain is not a new idea — it is the same impurity number, measured before and after, with the children weighted by size.

The Play Tennis case study — all 14 days

Day	Outlook	Temp	Humidity	Wind	Play?
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

The parent node, before any split

9 days Yes 5 days No n = 14

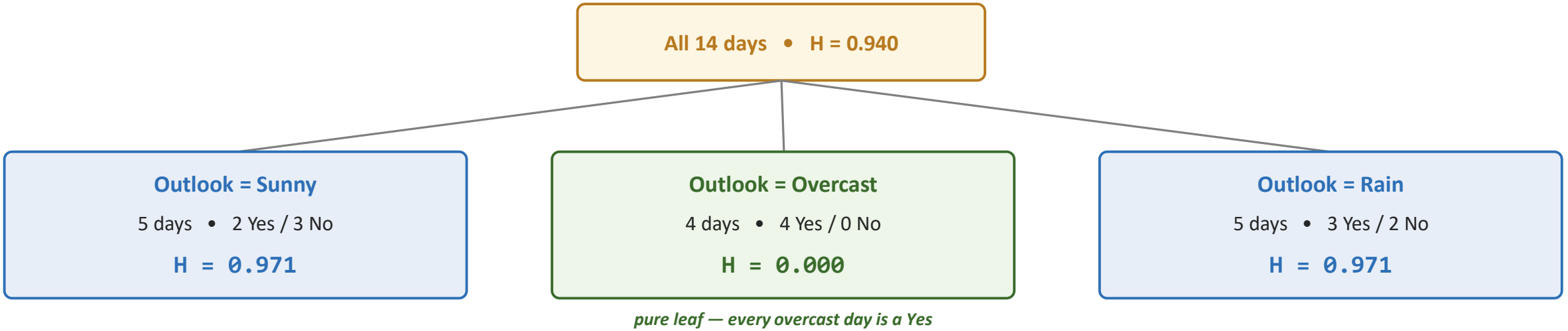
$H(S) = -(9/14)\log_2(9/14) - (5/14)\log_2(5/14)$

$H(S) = 0.940$

Four candidate root questions

- Outlook — three branches: Sunny, Overcast, Rain
- Temperature — three branches: Hot, Mild, Cool
- Humidity — two branches: High, Normal
- Wind — two branches: Weak, Strong

Worked example: Information Gain for Outlook



Weighted child entropy, then the gain

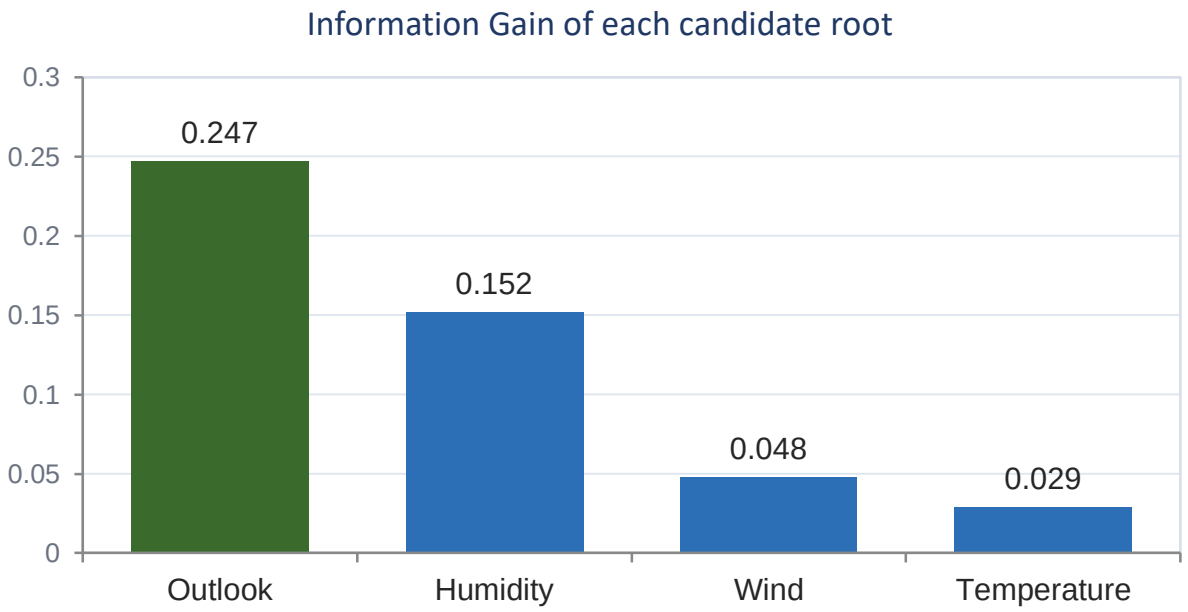
```
H_after = (5/14)(0.971) + (4/14)(0.000) + (5/14)(0.971)
        = 0.347 + 0.000 + 0.347   =   0.694
```

IG(Outlook) = 0.940 − 0.694 = 0.247

Why Outlook does so well

The Overcast branch is pure — 4 days, all Yes, entropy exactly 0. One of the three children is finished before the tree grows any further.

Comparing all four candidates: which feature wins the root?



Feature	Branches	H_after	IG
Outlook	5 / 4 / 5	0.694	**0.247**
Humidity	7 / 7	0.788	0.152
Wind	8 / 6	0.892	0.048
Temperature	4 / 6 / 4	0.911	0.029

Answering the video's question

Why is Outlook a better root than Wind?

Wind = Weak still leaves 6 Yes and 2 No — mixed. Wind = Strong is an exact 3/3 split, entropy 1.0, no information at all. Outlook hands us one finished branch immediately, so its gain is five times larger.

Outlook becomes the root — not because it looks sensible, but because 0.247 is the largest number in the table.

ID3: the tree-building loop

1	**Start**	Begin at the root with all training examples.
2	**Check for a leaf**	If every example here has the same class, make a leaf and stop.
3	**Score every candidate**	Compute Information Gain (or Gini gain) for each remaining feature.
4	**Choose the best**	Take the feature with the highest gain — this becomes the node's question.
5	**Branch and divide**	Create one branch per outcome and send the matching examples down it.
6	**Recurse**	Repeat from step 2 on every branch, independently.

repeat

The regression parallel

This is the decision-tree equivalent of the gradient-descent loop from the regression class. There we repeated an update until the cost stopped falling. Here we repeat a split until every node is pure.

The dangerous default

Notice step 2 is the only stopping rule. Left as written, ID3 grows until every leaf is pure — which on real data means memorising it.

Two decisions every tree algorithm must make

1. Which split?

SOLVED YESTERDAY

- Score every candidate feature with Information Gain or Gini gain.
- For a continuous feature, score every midpoint threshold too.
- Take the highest score. No ambiguity, no tuning.
- This is why the impurity measure had to come first.

A mechanical decision — the data answers it.

2. When to stop?

YOUR CHOICE

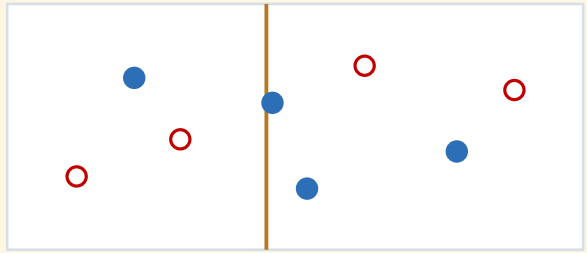
- Natural stop: every example in the node shares one class.
- Natural stop: no feature produces any gain at all.
- **But those alone give a fully grown, memorising tree.**
- So we impose limits: max_depth, min_samples_split, min_samples_leaf — or we prune afterwards.

A modelling decision — you answer it.

Too simple, about right, or memorising?

Too shallow

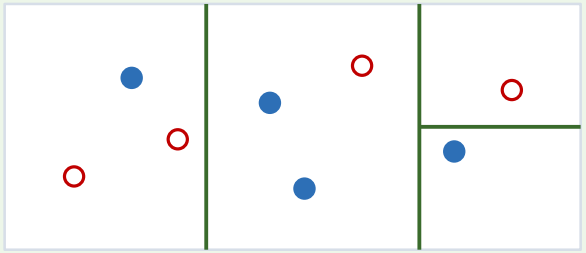
Underfitting



`depth 1 • 2 leaves`
`train 0.66  test 0.68`

About right

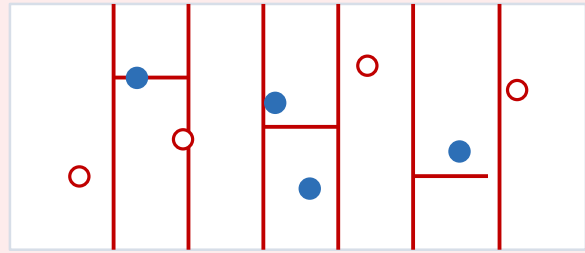
Good generalisation



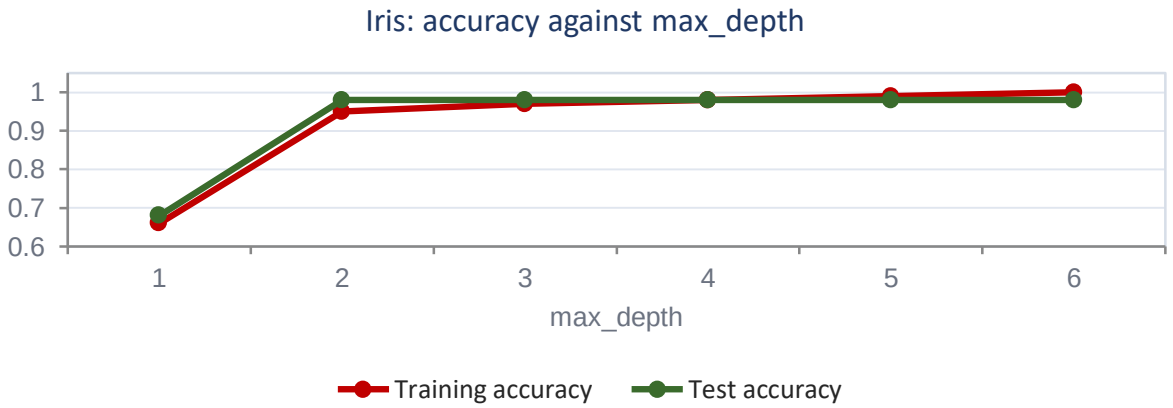
`depth 2 • 3 leaves`
`train 0.95  test 0.98`

Too deep

Overfitting



`depth 6 • 10 leaves`
`train 1.00  test 0.98`



Read the gap, not the test line

On Iris the test accuracy never actually falls — it sits at 0.98 from depth 2 onwards. What changes is the **gap**: training accuracy climbs to a perfect 1.00 while test stays flat.

Depth 6 buys 7 extra leaves and zero extra test accuracy. Occam's Razor says take the depth-2 tree.

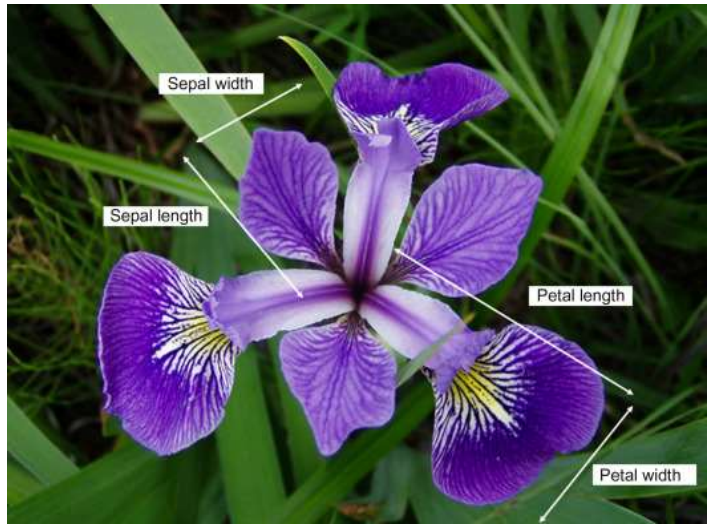
Python, first demo: load the data and split it

```python
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

iris = load_iris()
X = iris.data       # 4 measurements per flower
y = iris.target     # 3 species

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.33, random_state=42)

print(X_train.shape, X_test.shape)
```



Output

```
(100, 4)   (50, 4)
```

Python, second demo: grow the tree

CART (Classification and Regression Trees)

```python
from sklearn.tree import DecisionTreeClassifier
from sklearn import tree
import matplotlib.pyplot as plt

model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)

print(model.get_depth())      # 6
print(model.get_n_leaves())   # 10

plt.figure(figsize=(15, 10))
tree.plot_tree(model, filled=True,
               feature_names=iris.feature_names,
               class_names=iris.target_names)
plt.show()
```

$$\text{Weighted Gini} = \frac{N_L}{N} G_L + \frac{N_R}{N} G_R$$

$$\text{Gini decrease} = G_{\text{parent}} - G_{\text{after split}}$$

$$Gini_{\text{before}} = 0.6656$$

$$Gini_{\text{after}} = \frac{36}{100}(0) + \frac{64}{100}(0.5) \quad Gini_{\text{after}} = 0.32$$

$$\text{Gini decrease} = 0.6656 - 0.32 = 0.3456$$

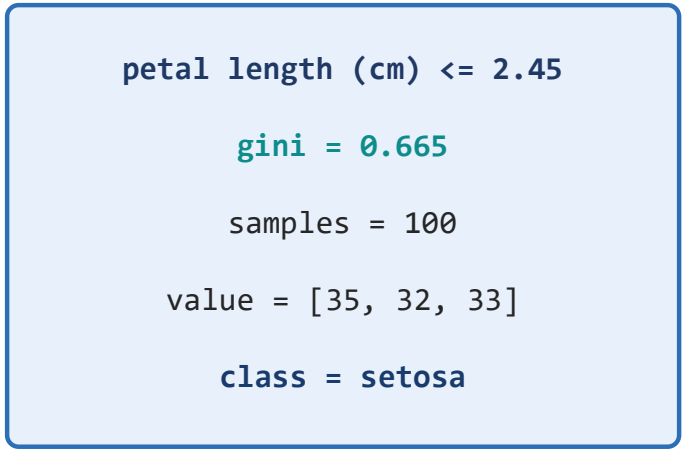
What you get back

`depth = 6` `leaves = 10`

`training accuracy = 1.000`

Every training flower classified correctly. That is not a success — it is the definition of a fully grown tree.

Reading the plotted tree: what is inside each box



The question	The split this node applies. Continuous feature → a learned threshold, not one you chose.
The impurity	Gini here, because that is the default. 0.0 would mean a pure leaf.
How many examples	The number of training rows that reached this node.
The class breakdown	Counts per class, in label order — [setosa, versicolor, virginica].
The prediction	The majority class. This is what a leaf would output.

Python, second demo: grow the tree

ID3

```python
from sklearn.tree import DecisionTreeClassifier
from sklearn import tree
import matplotlib.pyplot as plt

model = DecisionTreeClassifier(criterion='entropy', random_state=42)
model.fit(X_train, y_train)

print(model.get_depth())      # 6
print(model.get_n_leaves())   # 10

plt.figure(figsize=(15, 10))
tree.plot_tree(model, filled=True,
               feature_names=iris.feature_names,
               class_names=iris.target_names)
plt.show()
```

$$H(S) = -\left[\frac{36}{100}\log_2\frac{36}{100} + \frac{32}{100}\log_2\frac{32}{100} + \frac{32}{100}\log_2\frac{32}{100}\right]$$

$$= -(0.36\log_2 0.36 + 0.32\log_2 0.32 + 0.32\log_2 0.32)$$

$$= 0.531 + 0.526 + 0.526$$

$$\boxed{H(S) \approx 1.583}$$

$$H(L) = -(1\log_2 1) = 0$$

$$H(R) = -[0.5\log_2 0.5 + 0.5\log_2 0.5]$$

$$H_{\text{after}} = \frac{36}{100}H(L) + \frac{64}{100}H(R) = 0.64$$

$$IG = H(\text{parent}) - H(\text{after split})$$

$$IG = 1.583 - 0.64 \approx 0.943$$

Two ways to control tree size

Pre-pruning *(early stopping)*

- Constraints applied while the tree is being built.
- **max_depth, min_samples_split, min_samples_leaf, max_leaf_nodes**
- Cheap — the tree is never grown large in the first place.
- Risk: a split that looks weak now might have enabled a strong split below it. You will never find out.

Post-pruning

- Grow the full tree first, then remove branches that do not earn their keep.
- **ccp_alpha in sklearn • prune() in MATLAB**
- More expensive, but each branch is judged on what it actually contributed.
- This is what the literature means by cost-complexity pruning.

Python, third demo: pre-prune, then evaluate honestly

```python
from sklearn.metrics import accuracy_score
from sklearn.metrics import classification_report

pruned = DecisionTreeClassifier(
    max_depth=2, random_state=42)
pruned.fit(X_train, y_train)

y_pred = pruned.predict(X_test)   # unseen data

print(accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred,
      target_names=iris.target_names))
```

Test accuracy

0.98

`min_samples_split` = minimum size of the PARENT node

`min_samples_leaf` = minimum samples allowed in EACH resulting leaf

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

$$F_1 = 2\frac{Precision \times Recall}{Precision + Recall}$$

	prec	recall	f1	n
setosa	1.00	1.00	1.00	19
versicolor	0.94	1.00	0.97	15
virginica	1.00	0.94	0.97	16
accuracy			0.98	50

FP = model said YES, but truth was NO

FN = model said NO, but truth was YES

Python, fourth demo: genuine post-pruning with ccp_alpha

```python
# model is the fully grown tree from demo 2
path = model.cost_complexity_pruning_path(
                            X_train, y_train)

for a in path.ccp_alphas:
    m = DecisionTreeClassifier(
            random_state=42, ccp_alpha=a)
    m.fit(X_train, y_train)
    print(round(a, 5), m.get_n_leaves(),
          round(m.score(X_test, y_test), 3))
```

$$R_\alpha(T) = R(T) + \alpha|\widetilde{T}|$$

$$\alpha_{\text{eff}}(t) = \frac{R(t) - R(T_t)}{|T_t| - 1}$$

t = an internal node we are considering turning into one leaf
$R(t)$ = impurity cost if we make t one leaf
$R(T_t)$ = impurity cost of all current leaves below t
$|T_t|$ = number of leaves below t

$$R(t) = \frac{N_t}{N} \times Gini(t)$$

ccp_alpha	leaves	depth	test acc
0.00000	10	6	0.980
0.00944	6	4	0.980
0.00968	4	3	0.980
0.03380	**3**	**2**	**0.980**
0.25399	2	1	**0.680**
0.32087	1	0	**0.300**

The result worth pausing on

At α = 0.0338 post-pruning arrives at a 3-leaf, depth-2 tree — the same tree max_depth=2 produced by force. Two different routes, one answer. Past that point α gets too aggressive and accuracy collapses.

MATLAB, first demo: same task, different syntax

```
% Iris ships with MATLAB
load fisheriris
X = meas;              % 150 x 4 measurements
Y = species;           % class labels

% hold out 33% for testing
rng(42)
cv  = cvpartition(Y, 'HoldOut', 0.33);
Xtr = X(training(cv),:);   Ytr = Y(training(cv));
Xte = X(test(cv),:);       Yte = Y(test(cv));

% grow a full tree, then look at it
Mdl = fitctree(Xtr, Ytr);
view(Mdl, 'Mode', 'graph')
```

Line for line against Python

Python	MATLAB
`load_iris()`	`load fisheriris`
`train_test_split`	`cvpartition`
`model.fit(X,y)`	`fitctree(X,Y)`
`plot_tree(model)`	`view(Mdl,'graph')`

Two things that catch people out

- MATLAB indexes from 1; Python from 0.
- species is a cell array of strings, so compare with strcmp, not ==.
- fitctree uses Gini by default too — the same criterion as sklearn.

Python ↔ MATLAB: same workflow, different commands

Task	Python (scikit-learn)	MATLAB
Load the data	`load_iris()`	`load fisheriris`
Split train / test	`train_test_split(X, y, test_size=0.33)`	`cvpartition(Y,'HoldOut',0.33)`
Train a tree	`DecisionTreeClassifier().fit(X, y)`	`fitctree(X, Y)`
Choose the criterion	`criterion='entropy'`	`'SplitCriterion','deviance'`
Limit growth (pre-prune)	`max_depth=2`	`'MaxNumSplits',2`
Cut back (post-prune)	`ccp_alpha=0.0338`	`prune(Mdl,'Level',k)`
Predict	`model.predict(X_test)`	`predict(Mdl, Xte)`
Visualise	`tree.plot_tree(model, filled=True)`	`view(Mdl,'Mode','graph')`
Accuracy	`accuracy_score(y_test, y_pred)`	`mean(strcmp(Yhat, Yte))`